

SOC4190 BLOCKCHAIN BASICS & PLATFORMS

Bitcoin Transactions

Mode of teaching: **Online, asynchronous**

Instructor: Dr. Jalolliddin Yusupov

Email: j.yusupov@inha.uz

Where We Stopped

My video

Earlier we took this payment from block 968,391, checked its signature, and changed the amount by the smallest possible unit: rejected.
We know **who** agreed. We never looked at **what** was agreed.



One payment, six questions: today we open it.

Stop	Today's question	What we know
The coins	Where are the coins, before and after the payment?	— ❓
The cost	Who pays to send it, and where is that written?	— ❓
The bytes	What do its 225 bytes say?	— ❓
The lock	What stops anyone else from spending the coins?	— ❓
The address	How does Bitcoin make its 1… address?	— ❓
The size	What went wrong in 2014, and what does space in a block cost?	— ❓

A Story: Paying With Banknotes

Dilnoza has two banknotes in her pocket: **50,000** and **20,000** so'm. Lunch costs **60,000**.

She cannot tear a note in half. So she hands over **both** notes, 70,000 in total, and the cashier gives her back a **10,000** note.

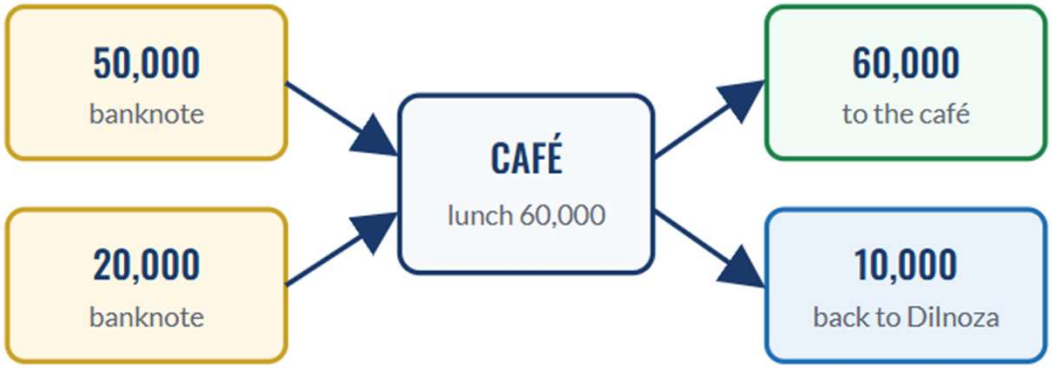
1	Old notes were handed over **whole**.
2	**New** notes came out: 60,000 for the café, 10,000 back to her.
3	"How much money does Dilnoza have?" = add up the notes in her pocket.



My video

Keep this picture

Bitcoin works exactly like this pocket of banknotes. The next slides give the three things their Bitcoin names.



notes go in whole · new notes come out · nothing is torn in half

Banknotes are spent whole; the rest comes back.

New Word: Satoshi

My video

📘 New word · Satoshi

Bitcoin's smallest unit: **1 BTC = 100,000,000 satoshis**, written "sat". Named after Satoshi Nakamoto. The ledger stores every amount as a whole number of satoshis — never a fraction.

Move the decimal point eight places: bitcoins to satoshis.

CALCULATE Write each amount of our payment in BTC.

```
1,000,000 sat = 0.01 BTC
  797,740 sat = 0.0079774 BTC
1,799,096 sat = 0.01799096 BTC
        1 sat = 0.00000001 BTC
```

1 Satoshi	=	0.00000001 BTC
10 Satoshi	=	0.00000010 BTC
100 Satoshi	=	0.00000100 BTC
1,000 Satoshi	=	0.00001000 BTC
10,000 Satoshi	=	0.00010000 BTC
100,000 Satoshi	=	0.00100000 BTC
1,000,000 Satoshi	=	0.01000000 BTC
10,000,000 Satoshi	=	0.10000000 BTC
100,000,000 Satoshi	=	1.00000000 BTC

There are 8 zeros

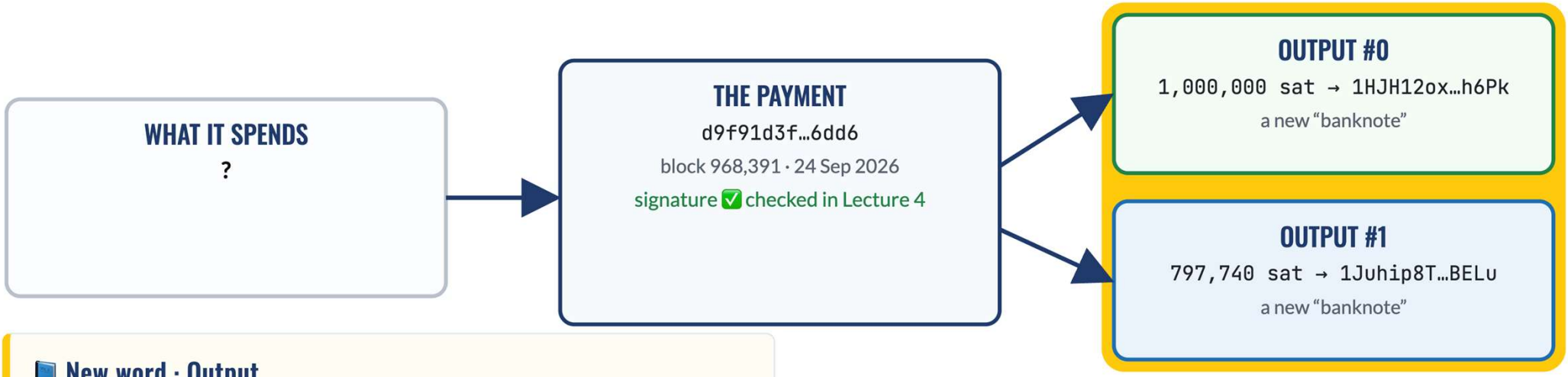
New Word: Output

My video

A transaction creates outputs: amounts with locks.

Our payment created two outputs: **1,000,000 sat** locked to `1HJH12oxKr3JJMeMSFSmYhGMoyocEGh6Pk` and **797,740 sat** locked to `1Juhip8TnfZeHUQwuE4pDFtGtPZQNmBELu`.

Outputs are numbered from 0, in the order they are written.



🟦 New word · Output

A new "banknote" created by a transaction: an **amount** in satoshis and a **lock** that says who may spend it — in practice, an address.

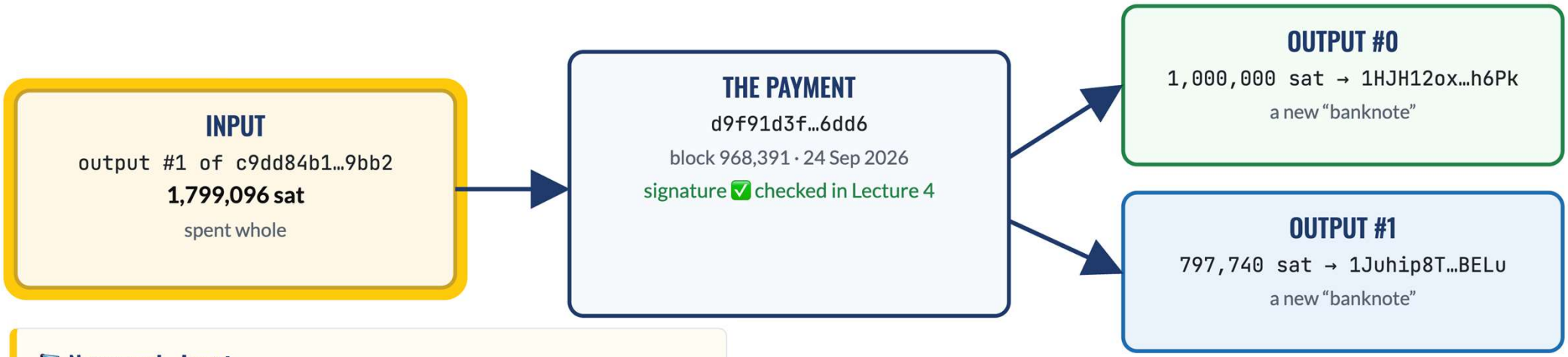
New Word: Input

An input spends an earlier output, whole.

Our payment has one input: output #1 of payment `c9dd84b125…2f9bb2`, written into block 968,232 on 23 Sep 2026. It held **1,799,096 sat**.

The input does not say "take 1,000,000 from my account". It says "I spend that whole banknote".

My video



📘 **New word · Input**

A pointer to an output of an **earlier** transaction, which this transaction spends — **whole**, like a banknote handed over.

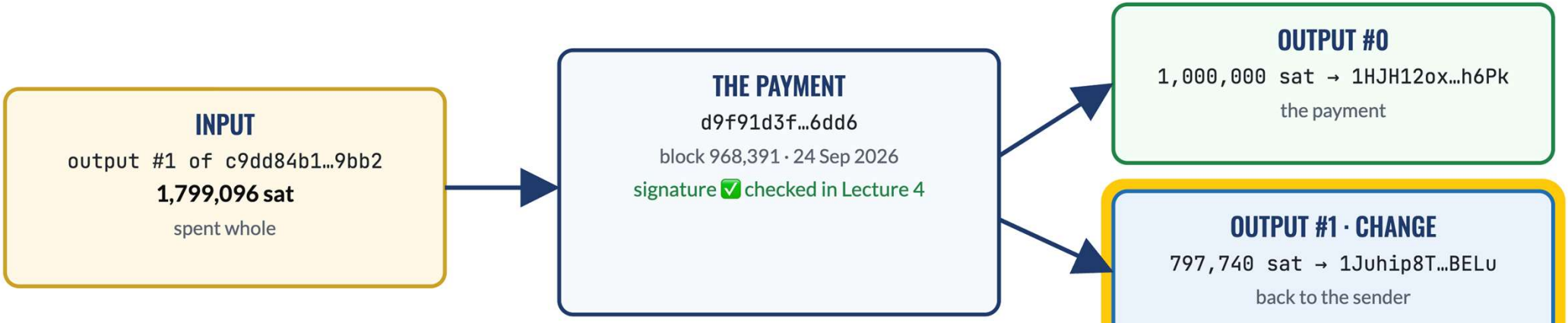
New Word: Change

My video

Who decides?

Nobody marks an output as "change". The wallet builds it automatically; on the chain it is just another output. We recognised it because it returns to the sender.

The input held 1,799,096 sat, the payment needed 1,000,000. So output #1 sends **797,740 sat** back to `1Juhip8Tnf…NmBELu` — the very address the input came from.



New word · Change

An output that sends the rest of a spent banknote back to the payer — usually to an address of the payer's own wallet.

What is not paid comes back as change — as a new output.

New Word: Transaction Fee

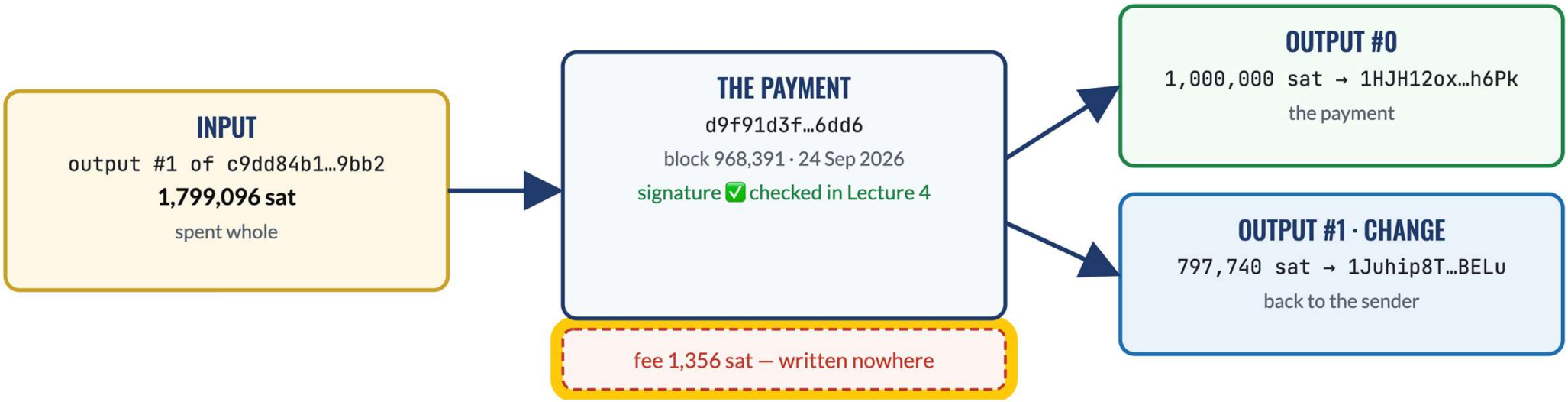
My video

New word · Transaction fee

What a transaction leaves to the miner who writes it into a block: everything its inputs bring in that its outputs do not claim. No field in the transaction holds it.

CALCULATE inputs − outputs = fee

```
1,799,096 - 1,000,000 - 797,740
= 1,356 sat = 0.00001356 BTC
```



The fee is what the outputs leave unclaimed: written nowhere, paid anyway.

⏭ What else a miner earns for a block — the mining lecture.

No balances on Bitcoin: only unspent outputs, which the wallet adds up.

My video

New word · UTXO — unspent transaction output

An output that no input has spent yet. Every bitcoin that exists right now sits in some UTXO. The list of all of them, which every node keeps, is the **UTXO set**.

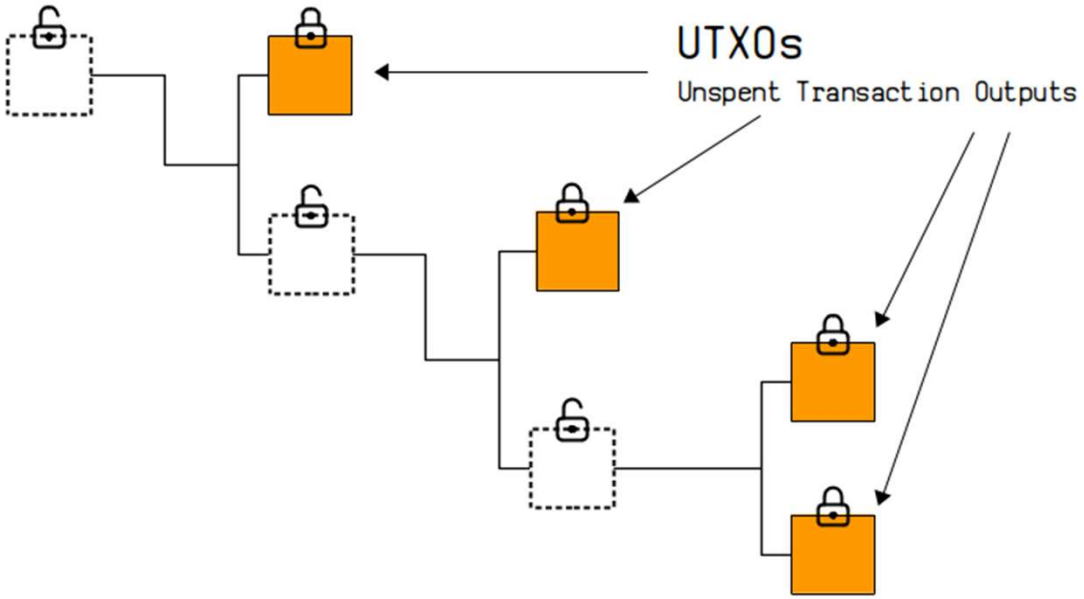
New Word: UTXO

EVERY UNSPENT OUTPUT LOCKED TO YOUR ADDRESS

50,000 sat	120,000 sat	8,000 sat
from payment A	from payment B	from payment C

your wallet shows: 178,000 sat

· the "balance" is a sum the wallet computes



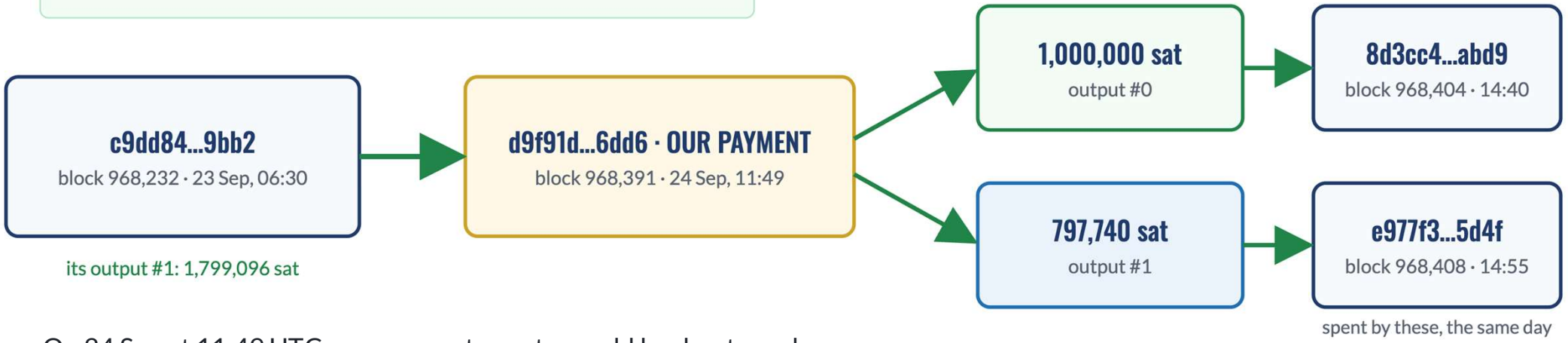
Lecture 1: a balance is not stored anywhere — it is **calculated** from the records. Bitcoin takes this literally. There is no line "address X has 0.5 BTC". Your wallet finds every UTXO locked to your addresses and adds them up.

Every Coin Has a History

My video

Follow any coin backwards

Every input names the output it spends; that output was created by an earlier payment, whose inputs name earlier outputs … all the way back to the first transaction of some block, where new coins appear.



On 24 Sep at 11:49 UTC our payment spent one old banknote and made two new ones. By 14:55 the same day **both** new outputs had been spent again: they are no longer UTXOs.

⏭ How new coins appear in a block — the mining lecture.

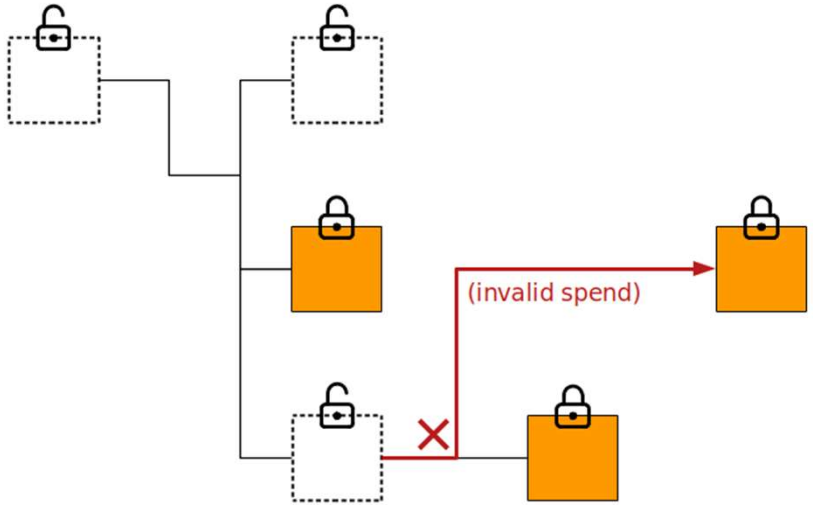
Every output is born in one payment and dies in another.

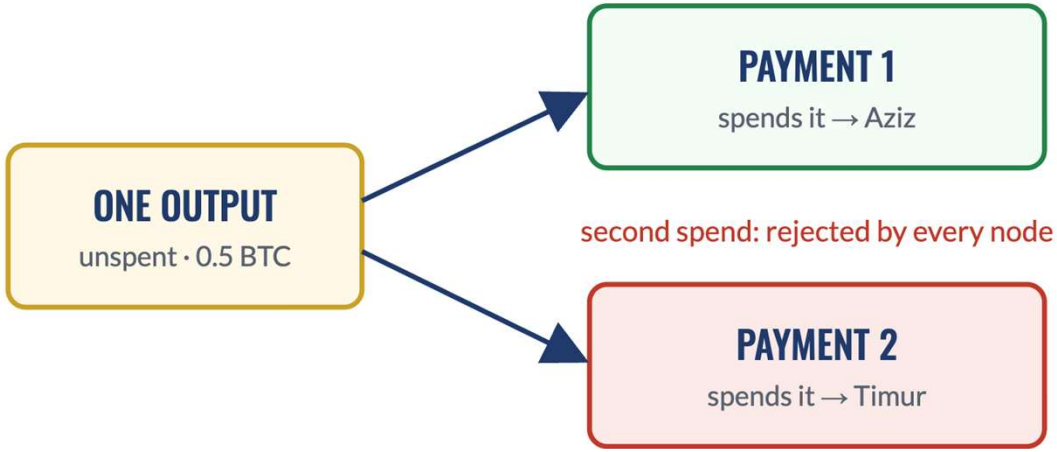
Double Spending, Caught

My video

Lecture 1's problem: digital money can be copied and spent twice.
Bitcoin's answer is now simple to state.

1	An output can be spent **once**.
2	Every node keeps the UTXO set.
3	A payment whose input points to an output that is not in the set — already spent, or never existed — is rejected by every node.
4	If two payments try to spend the same output, the one written into a block first wins; the other is dropped everywhere.





⏭ Who decides which payment is written first — the mining lecture.

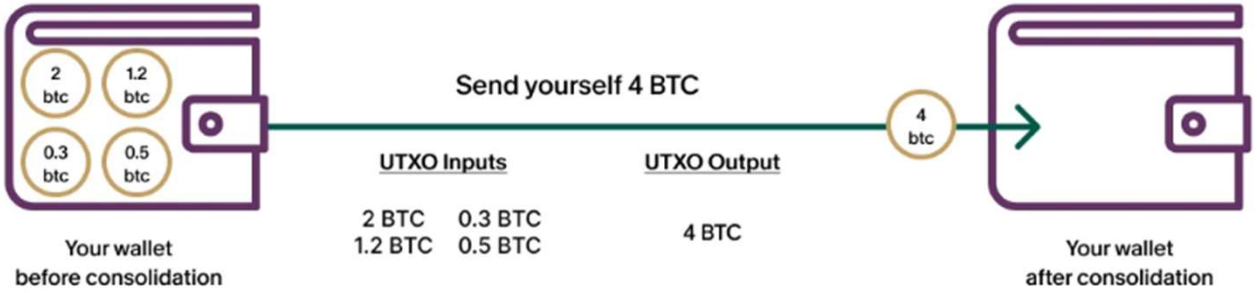
An output is spent once; every node checks the UTXO set.

Many Notes Into One

My video

Payment	`0a1ddd0163…2ab097`
Block	968,390
Inputs	**30** UTXOs, from 1,690 to 1,082,850 sat, locked to 30 different addresses
Output	**1**: 6,748,498 sat, to one new address
Fee	6,756,736 − 6,748,498 = **8,238 sat**

Many inputs, one output: change works in both directions.



The fee does not follow the amount

This payment moved 0.06748498 BTC and paid 8,238 sat; ours moved 0.01 and paid 1,356. Fees follow **size**, not value — later today.

Why would anyone do this?

Thirty small banknotes in thirty pockets are heavy: spending them later means thirty inputs in some future payment. Merging them now, while fees are low, leaves one tidy UTXO. Who did it, the chain does not say.

War Story: The 83-Bitcoin Fee

Date	2023-11-23, block 818,087
Payment	`b5a2af5845…48f9aa`
Input	1 · 139.42495946 BTC
Output	1 · 55.76998378 BTC
Fee	**83.65497568 BTC** = 8,365,497,568 sat
	$3.15 million at the time

Lesson

The fee is whatever the outputs forget to claim. No field, no warning, no undo. Never build payments by hand with real coins, and never switch off the wallet's large-fee warning.

What happened next

The owner said a faulty wallet program had built the payment. On 30 Nov 2023 Antpool, whose computers wrote that block, offered to return the fee — if the owner proved ownership by **signing a message with the private key**.

Compare with our payment

Ours was 225 bytes and paid 1,356 sat. This one was about half the size of ours and paid over six million times more.

My video



Whatever the outputs forget to claim becomes the fee — and a fee cannot be undone.

Coins and fee explained. Next: the bytes.

Two Questions Gone

Stop	Today's question	What we know	
The coins	Where are the coins, before and after the payment?	in outputs: new “banknotes”, each locked to an address and spent whole — UTXOs	✅
The cost	Who pays to send it, and where is that written?	the fee = inputs − outputs = 1,356 sat, written nowhere	✅
The bytes	What do its 225 bytes say?	—	❓
The lock	What stops anyone else from spending the coins?	—	❓
The address	How does Bitcoin make its 1… address?	—	❓
The size	What went wrong in 2014, and what does space in a block cost?	—	❓

My video

What do the 225 bytes say?

INPUT
`output #1 of c9dd84b1…9bb2`
1,799,096 sat
spent whole

THE PAYMENT
`d9f91d3f…6dd6`
block 968,391 · 24 Sep 2026
signature ✅ checked in Lecture 4
A median Bitcoin transaction size is approximately **225 bytes**

fee 1,356 sat — written nowhere

OUTPUT #0
`1,000,000 sat → 1HJH12ox…h6Pk`
the payment

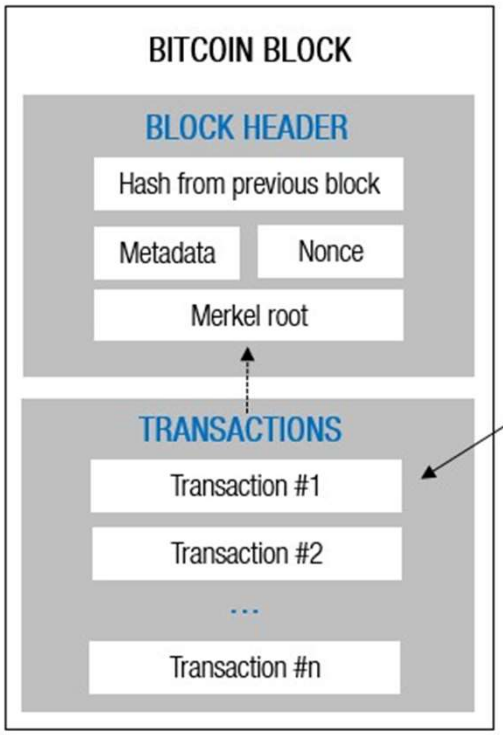
OUTPUT #1 · CHANGE
`797,740 sat → 1Juhip8T…BELu`
back to the sender

The coins are outputs, spent whole; the fee is what they leave behind. Now the payment itself: what is actually written?

New Word: Raw Transaction

A payment is 225 bytes. Everything we said about it is in there.

My video



📘 New word · Raw transaction

A transaction exactly as nodes send it to each other and as blocks store it: a row of bytes. Ours is 225 bytes, shown here as 450 hexadecimal characters.

```
0200000001b29b2f64d67b4a0da277ca2ef50e3847c28cd4023ca8933d8fc0ae
25b184ddc9010000006a47304402207c58a73023515ce7e760843e4eba95f305
bd3a3209a119e5f08dfca55514b3dd022038c29f020e6330195f788c58ec1975
350338fe75b04e4f737e242e2adb79cb9a0121026c22163f202760a7189cac51
704b6f21149c208a7a258b752ff24b28b0a8f405ffffffff0240420f00000000
001976a914b2c670c276afb59fd446710cb6f6c98202249e5c88ac2c2c0c0000
0000001976a914c471c2ac102c39809c341a1a71b1c7b0dd6681d788ac000000
00
```

version | how many inputs | previous payment's ID | output index | length byte | the key | 4 extra bytes | how many outputs | amount | the lock | last 4 bytes

The Map of a Transaction

My video

Two kinds of counting byte

Before the inputs, 1 byte says how many inputs follow; before the outputs, 1 byte says how many outputs. Before the key and before each lock, 1 byte says how long it is. A bigger count would simply use more bytes.

the input = previous payment's ID 32 · output index 4 · length byte 1 · the key 106 · 4 extra bytes

each output = amount 8 · length byte 1 · the lock 25

OUR PAYMENT · 225 BYTES, IN ORDER, LEFT TO RIGHT



The first and the last 4 bytes

The first 4: the **version** (here 2), the rules the payment follows — like the block's version. The last 4: the earliest block this payment may enter; 0 means “any”.

Version, inputs, outputs, last 4 bytes: every transaction has this shape.

⏭ Payments that wait for a future block — the wallets lecture.

New Word: Little-Endian

New word · Little-endian

Written smallest byte first. To read the number, read the bytes backwards.

CALCULATE The 8 amount bytes of output #0, as they sit in the payment:

```
as stored     40 42 0f 00 00 00 00 00
backwards     00 00 00 00 00 0f 42 40
hexadecimal   0xf4240
decimal       1,000,000 sat ✓
```

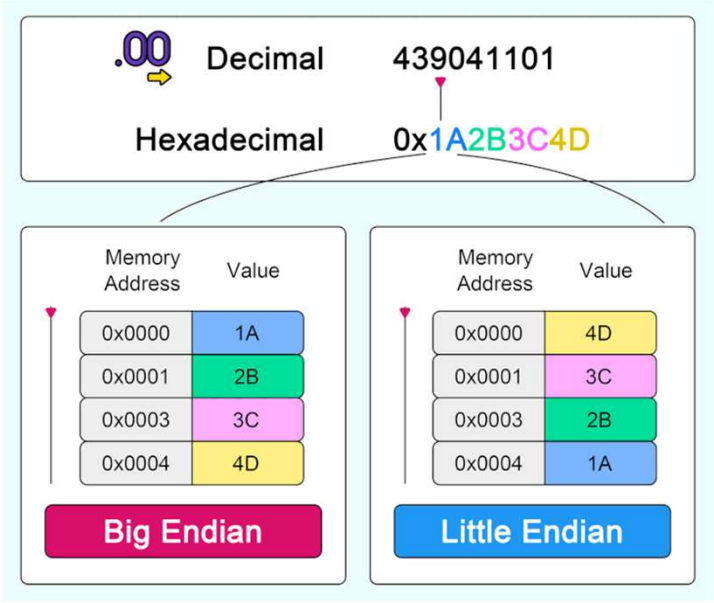
Why backwards?

Most computers keep numbers in memory this way; Bitcoin simply writes the bytes as the computer holds them. The previous payment's ID is stored backwards too.

Read Bitcoin's numbers backwards, byte by byte.

Your turn: output #1's amount is stored as `2c 2c 0c 00 00 00 00 00`. Which number is it?

My video



Decoding the Input

My video

Two old friends

The 71-byte signature and the 33-byte public key are exactly what Lecture 4 checked. They live inside the input.

Part	Bytes	In our payment
previous payment's ID	32	stored `b29b2f64d67b4a0d…` → read backwards: `c9dd84b125aec08f…2f9bb2`
output index	4	`01000000` → 1: "the output numbered 1"
length byte	1	`6a` → 106 bytes follow
the key that opens the coin	106	1 + **71**-byte signature + 1 + **33**-byte public key
4 extra bytes	4	`ffffffff` — the usual value, nothing special

```
0200000001b29b2f64d67b4a0da277ca2ef50e3847c28cd4023ca8933d8fc0ae
25b184ddc9010000006a47304402207c58a73023515ce7e760843e4eba95f305
bd3a3209a119e5f08dfca55514b3dd022038c29f020e6330195f788c58ec1975
350338fe75b04e4f737e242e2adb79cb9a0121026c22163f202760a7189cac51
704b6f21149c208a7a258b752ff24b28b0a8f405ffffffff0240420f00000000
001976a914b2c670c276afb59fd446710cb6f6c98202249e5c88ac2c2c0c0000
0000001976a914c471c2ac102c39809c341a1a71b1c7b0dd6681d788ac000000
00
```

An input = which output it spends + the key that opens it.

Decoding the Outputs

An output = 8 bytes of amount + a lock.

My video

Output	Amount (8 bytes, backwards)	The lock (25 bytes)
#0	`40420f0000000000` → 1,000,000 sat	`76a914b2c670c276afb59fd446710cb6f6c98202249e5c88ac` contains the 20 bytes of the receiver's address
#1	`2c2c0c0000000000` → 797,740 sat	`76a914c471c2ac102c39809c341a1a71b1c7b0dd6681d788ac` contains the 20 bytes of the sender's address

No addresses inside

The payment never writes "1HJH12ox…". It writes 25 bytes of lock. The explorer turns the 20 bytes in the middle into an address for us.

Where is the fee?

Nowhere: two amounts here, one amount in the earlier output. 1,799,096 − 1,000,000 − 797,740 = 1,356.

```
0200000001b29b2f64d67b4a0da277ca2ef50e3847c28cd4023ca8933d8fc0ae
25b184ddc9010000006a47304402207c58a73023515ce7e760843e4eba95f305
bd3a3209a119e5f08dfca55514b3dd022038c29f020e6330195f788c58ec1975
350338fe75b04e4f737e242e2adb79cb9a0121026c22163f202760a7189cac51
704b6f21149c208a7a258b752ff24b28b0a8f405ffffffff0240420f00000000
001976a914b2c670c276afb59fd446710cb6f6c98202249e5c88ac2c2c0c0000
0000001976a914c471c2ac102c39809c341a1a71b1c7b0dd6681d788ac000000
00
```

New Word: Transaction ID

My video

New word · Transaction ID (TXID)

The name of a transaction: SHA-256 of its raw bytes, then SHA-256 again — written backwards, 64 hexadecimal characters. Exactly the machine of Lecture 4's block hash.

LIVE DEMO The 225 bytes in a file, hashed twice:

```
$ python3 txid.py payment.hex
225 bytes  txid: d9f91d3f4d39a09f91c47070631382df3c3c9808380be5aff4b8dc6211706dd6
$ python3 txid.py payment_plus1.hex
225 bytes  txid: 3babb094ea8d6aae86353b91038c92182edb8a34527c41f3ea6bab41fc1fed1e
```

The first ID is the one every explorer shows. Nobody assigns it: every node computes it from the bytes.

One satoshi more, and the ID is a different one: only **3** of 64 characters still match — Lecture 4's avalanche.

Why it matters

An input names the output it spends by this ID and an output number. Change one byte of a payment and every later payment that points to it would point to nothing.

fundamental security mechanism of cryptocurrency transactions

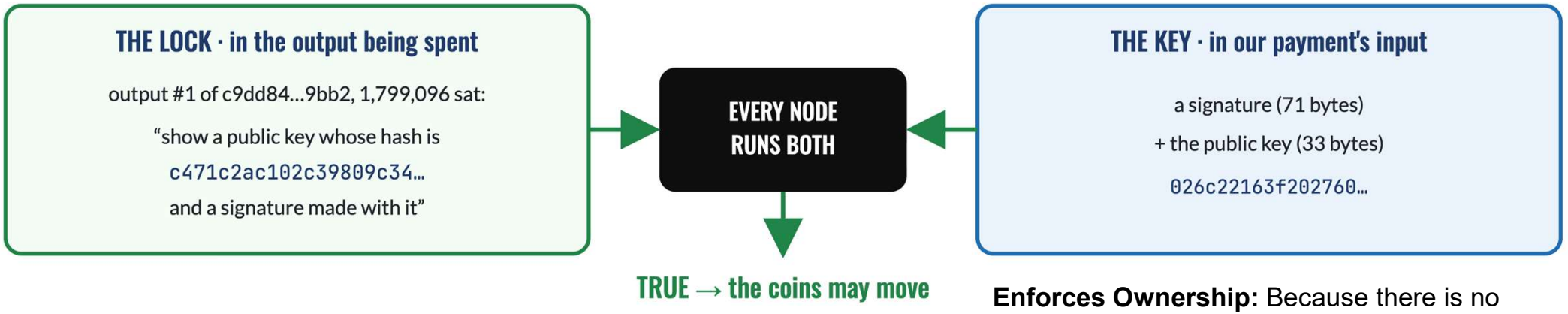
Locks and Keys

My video

New word · Unlocking script

The key inside every input: the data that satisfies the lock of the output being spent — for our payment, a signature and a public key.

Outputs carry locks, inputs carry keys; every node tries the key in the lock.



New word · Locking script

The lock inside every output: a tiny program that states what must be shown to spend it.

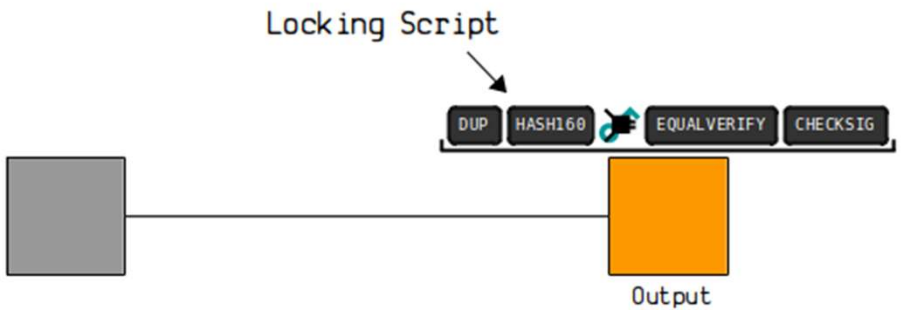
Enforces Ownership: Because there is no central bank or authority to verify who owns what, public-key cryptography serves as digital security. Only the person who holds the correct private key can generate the valid signature (the key) required to unlock the funds

New Words: Script and Opcode

My video

THE FOUR OPCODES OF OUR LOCK

76 OP_DUP	copy the top item
a9 OP_HASH160	replace the top item by its 20-byte hash
88 OP_EQUALVERIFY	the two top items must be equal, else stop: FALSE
ac OP_CHECKSIG	does the signature fit this public key and this payment?



New word · Script

Bitcoin's tiny programming language for locks and keys. No loops, no files, no internet — on purpose: every node must be able to run it quickly and safely, for every input.

New word · Opcode

One instruction of Script, written as one byte. Its names begin with OP_.

Script is a tiny language with no loops: locks are small programs.

The P2PKH Lock, Byte by Byte

Pay-to-Public-Key-Hash (P2PKH)

P2PKH: show the public key behind the hash, and sign with it.

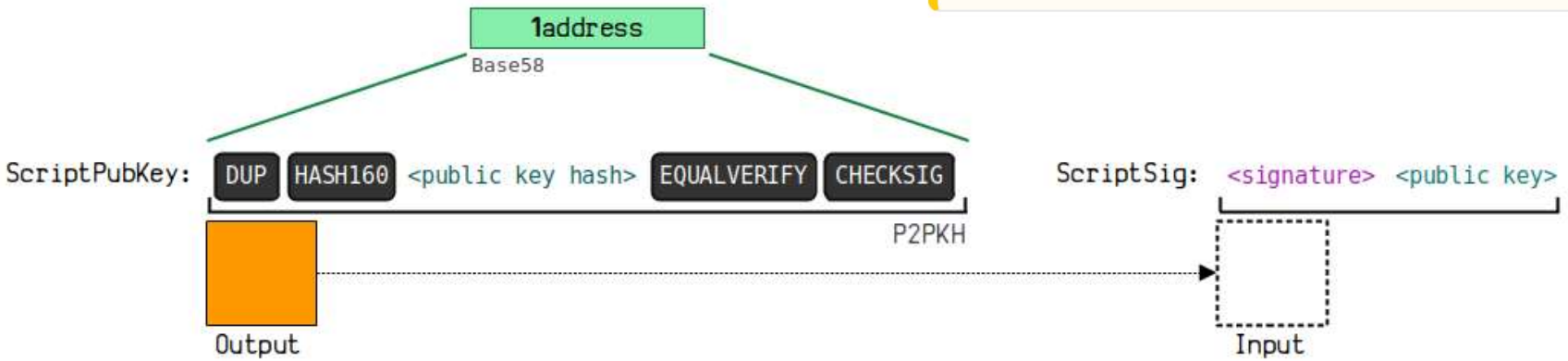
```
76 a9 14 c471c2ac102c39809c341a1a71b1c7b0dd6681d7 88 ac
```

My video

`76`	OP_DUP
`a9`	OP_HASH160
`14`	"the next 20 bytes are data" (14 in hexadecimal = 20)
`c471c2ac…`	the 20-byte hash of the owner's public key
`88 ac`	OP_EQUALVERIFY · OP_CHECKSIG

📘 New word · P2PKH — pay to public key hash

The classic Bitcoin lock: "show a public key that hashes to these 20 bytes, and a signature made with its private key".



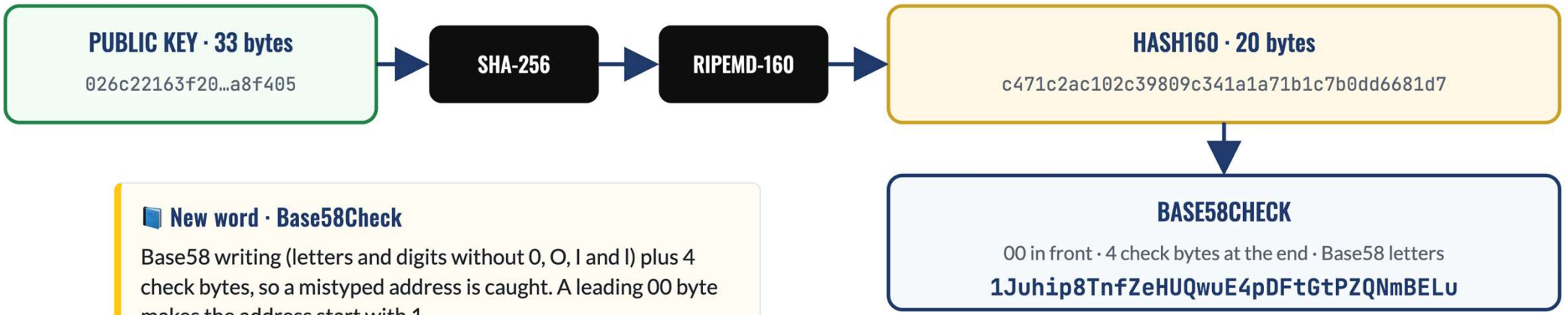
How Bitcoin Makes Its Address

My video

New word · HASH160

SHA-256, then **RIPEMD-160** — another, older hash function whose fingerprint is 160 bits (20 bytes). Short enough to write, still impossible to reverse.

the sender of our payment:
the same 20 bytes sit inside the
lockof the coin it spent



New word · Base58Check

Base58 writing (letters and digits without 0, O, I and l) plus 4 check bytes, so a mistyped address is caught. A leading 00 byte makes the address start with 1.

A 1… address is the public key's HASH160, written in Base58Check.

DRAW: Running P2PKH, Step by Step



My video

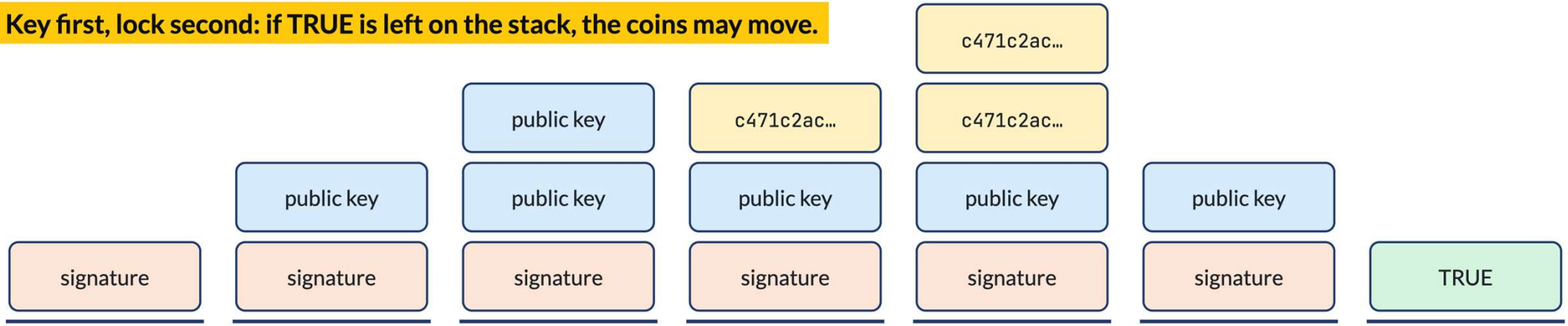
DRAW The key's two items go on first, then the lock runs. Draw the seven stacks before you look.

New word · Stack

A pile where you can only put an item on top, or take the top one off — like a stack of plates. Script works on one stack.

1. push	2. push	3. OP_DUP	4. OP_HASH160	5. push	6. OP_EQUALVERIFY	7. OP_CHECKSIG
the signature	the public key	copy the top	hash the top	the hash in the lock	equal? remove both	signature fits?

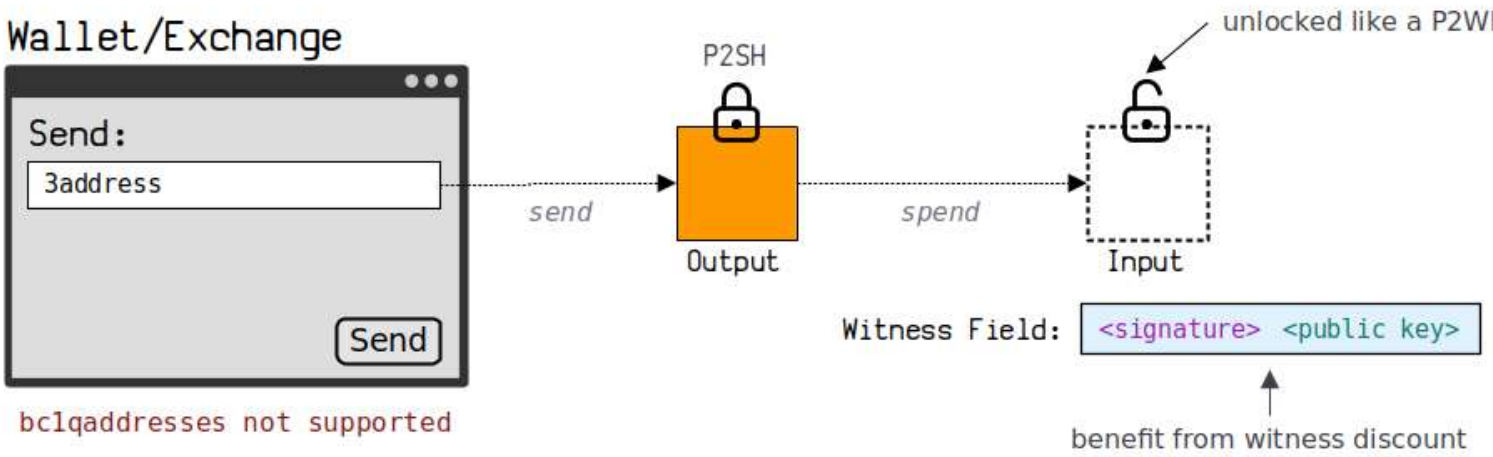
Key first, lock second: if TRUE is left on the stack, the coins may move.



P2SH locks coins to the hash of a script; the spender reveals the script.

New Word: P2SH

My video



Why hide the script?

The payer only needs a short address. The long conditions — here, two of three people must agree — are revealed, and paid for, only by whoever spends.

Address	`3CTkLec3ubxkyUV2NMqLH8Uv3uQeNDfJLG`
Spent in	`ac42ccf5cb…cbe290`, block 968,389
The hidden script	105 bytes: **any 2 of 3** listed public keys must sign
The input showed	2 signatures + the script itself

📘 New word · P2SH — pay to script hash

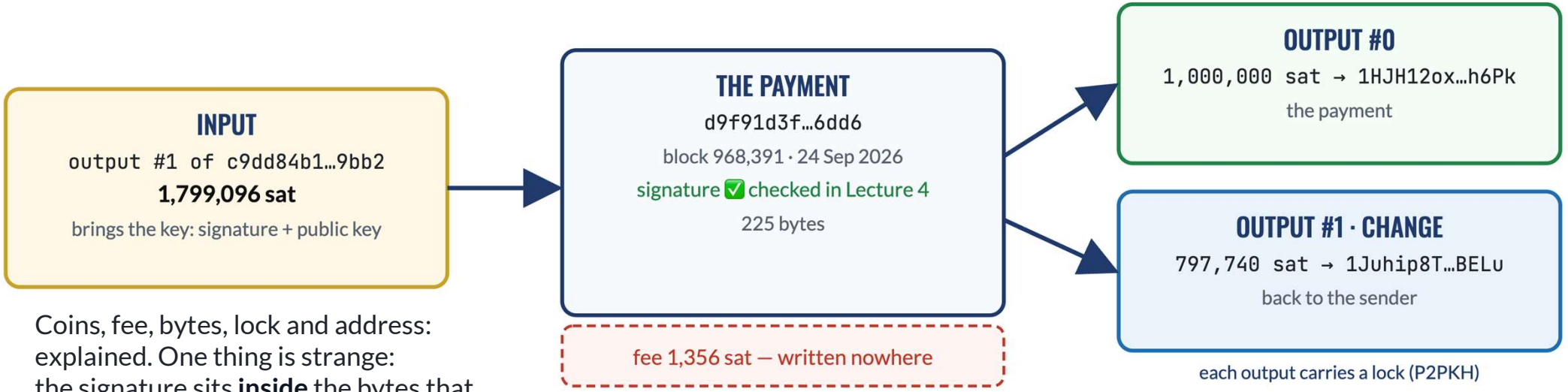
A lock that holds only the HASH160 of a longer script. To spend, the input must show that script — and satisfy it. Addresses of this kind start with 3.

⏭ Wallets that need several keys — the wallets lecture.

One Question Left

Stop	Today's question	What we know	
The bytes	What do its 225 bytes say?	version · 1 input · 2 outputs · last 4 bytes; its ID = SHA-256 twice	✅
The lock	What stops anyone else from spending the coins?	a locking script: P2PKH — a public-key hash, opened by a signature	✅
The address	How does Bitcoin make its 1… address?	SHA-256, then RIPEMD-160 (HASH160), then Base58Check	✅
The size	What went wrong in 2014, and what does space in a block cost?	—	❓

My video



Coins, fee, bytes, lock and address: explained. One thing is strange: the signature sits **inside** the bytes that make the transaction ID.

The ID covers the signature. Is that a problem?

New Word: Transaction Malleability

My video

📘 New word · Transaction malleability

Someone other than the payer can change how a signature is written — without breaking it — and so change the transaction ID before the payment is in a block.

Why it is possible

The signature cannot sign itself: it is made *from* the payment, so it is left out of what is signed. But it is put back *into* the bytes that are hashed for the ID.

Same payment, two names

Same inputs, same outputs, same amounts — a different ID. Anyone tracking the payment by its ID would think it vanished.

What we did	took our signature's second number s and replaced it by n − s (n: about the number of possible private keys, Lecture 5)
Still valid?	**yes** — the same signature check accepts it
Signature size	71 → 72 bytes
New ID	e6f51ad13ecb…b9f254df

If the signature is inside the ID, the ID can be changed without the payer.

War Story: Mt. Gox, February 2014

Who (Tokyo)	Mt. Gox — then the largest exchange (a company where people buy and sell coins)
7 Feb 2014	stopped all bitcoin withdrawals
10 Feb 2014	blamed transaction malleability: its system had been tricked into paying twice
End of Feb 2014	closed; about 850,000 bitcoins reported missing

My video

Respect

Many ordinary people lost savings they had trusted to one company. It took years of court cases before any money was returned.

Engineering lessons

1. Never track a payment only by its ID before it is in a block.
2. A real flaw is not automatically the cause of a loss — check the evidence. 3. Coins held by an intermediary are only as safe as that intermediary.

What the researchers found

Decker and Wattenhofer (ETH Zürich) watched the network for a year. The flaw was real, but before the closure they found almost no use of it: at most about 386 bitcoins could have been taken this way.

"Bitcoin Transaction Malleability and MtGox", arXiv:1403.6676, 2014.

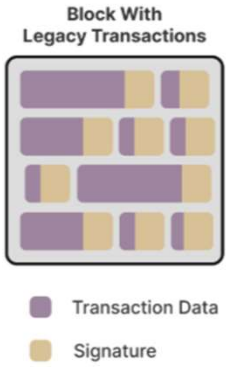
The flaw was real; the evidence said it was not the main cause.

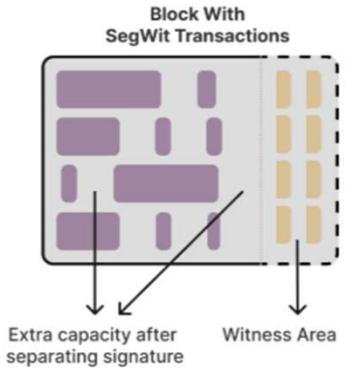
New Word: SegWit

My video

📘 New word · SegWit — segregated witness

An upgrade that moves signatures and public keys out of the bytes hashed for the transaction ID, into a separate section called the **witness**. Malleability of those inputs is gone.





+ 2 bytes (00 01) that say "this transaction has a witness section"

WHO PAYS WHOM · 113 bytes

version · input (its unlocking script left empty) · outputs · last 4 bytes

THE WITNESS · 108 bytes

signature + public key, in their own section

hashed → **transaction ID**: the witness cannot touch it

still sent, and still checked by every node

Switched on	block 481,824, 2017-08-24
A real SegWit payment	`0db3220c46…a0d854:` 223 bytes = 113 + 2 + 108 witness

Its address starts with bc1

SegWit outputs are written in a new alphabet, **bech32**: lower-case letters and digits, with a stronger typo check. This payment paid `bc1qm5ntyg…2nzpc7`.

SegWit keeps signatures out of the transaction ID.

New Words: Weight and Virtual Byte

My video

📘 New word · Weight

How much room a transaction takes in a block: every ordinary byte counts 4, every witness byte counts 1. Weight = 3 × (bytes without witness) + (all bytes). A block may hold at most **4,000,000**.

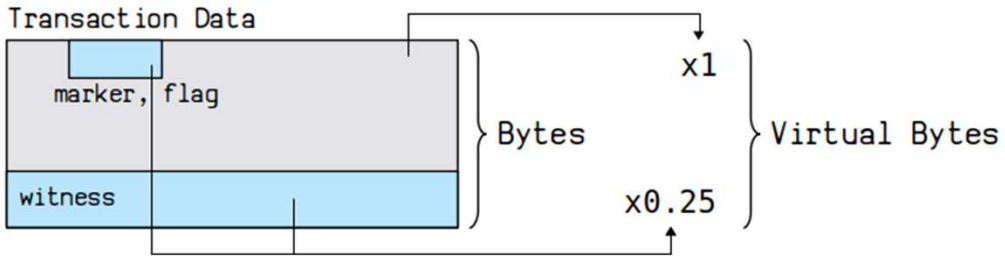
📘 New word · Virtual byte (vB)

Weight ÷ 4. For a payment without a witness, virtual bytes = bytes.

CALCULATE Two real payments

```
ours:    3 × 225 + 225 = 900 → 225 vB
SegWit:  3 × 113 + 223 = 562 → 141 vB
```

Almost the same number of bytes (225 vs 223), but the SegWit one takes 63 % of the room.



Room in a block is measured in weight; witness bytes weigh a quarter.

New Words: Fee Rate and Mempool

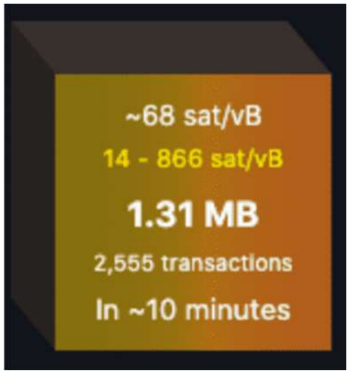
My video

📘 New word · Mempool

The waiting room: valid payments that every node has heard of but no block holds yet. mempool.space is named after it.

📘 New word · Fee rate

The fee divided by the size: satoshis per virtual byte (sat/vB). Miners fill the limited room of a block with the highest fee rates first.



CALCULATE fee ÷ vB

```
ours     1,356 ÷ 225 = 6.03 sat/vB   (24 Sep)
SegWit   980 ÷ 141 = 6.95 sat/vB
today's suggestion:   1 sat/vB
```

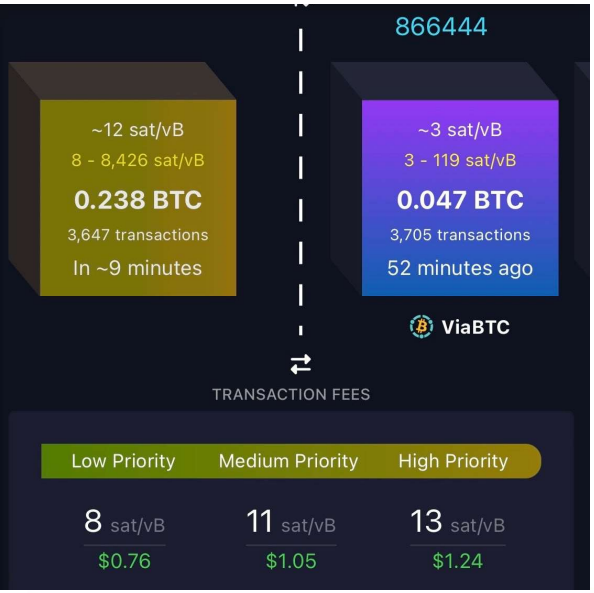
So what does a payment cost?

Not a share of the amount: its size times the fee rate of the moment. A 30-input payment costs more than a 1-input one, whatever it moves.

Fee = size × fee rate: space in a block is what you pay for.

New Words: Fee Rate and Mempool

My video



INPUT

`output #1 of c9dd84b1…9bb2`

1,799,096 sat

brings the key: signature + public key

THE PAYMENT

`d9f91d3f…6dd6`

block 968,391 · 24 Sep 2026

signature ✅ checked in Lecture 4

225 bytes · 225 vB · 6.03 sat/vB

fee 1,356 sat — written nowhere

OUTPUT #0

`1,000,000 sat → 1HJH12ox…h6Pk`

the payment

OUTPUT #1 · CHANGE

`797,740 sat → 1Juhip8T…BELu`

back to the sender

each output carries a lock (P2PKH)

One payment: banknotes in, banknotes out, a lock on each, and a fee written nowhere.



10/17/2026

SOC4190 2026 Yusupov